

Efficient Large Integers Arithmetic by Adopting Squaring and Complement Recoding Techniques

Chia-Long Wu*, Der-Chyuan Lou, and Te-Jen Chang

**Department of Aviation and Communication Electronics
Chinese Air Force Institute of Technology
chialongwu@seed.net.tw*

*Department of Electrical Engineering
Chung Cheng Institute of Technology, National Defense University
dclou@ccit.edu.tw*

Abstract

The efficient squaring algorithm plays an important role in large number arithmetic. All multiplication algorithms can be used for squaring large integers, but their performance can be greatly improved by using the standard squaring algorithm. Complement recoding method can be divided into two groups: 1's complement recoding method and 2's complement recoding method. In this paper, we propose an efficient squaring algorithm by combining 1's complement recoding method with standard squaring algorithm to fast evaluate the squaring. The computational complexity of the proposed squaring method is approximately $\frac{k^2}{8}$ multiplications, which is faster than that of the standard squaring method $\frac{k^2}{4}$ multiplications, where k is the bit-length of X . We can therefore efficiently speed up the overall performance of squaring arithmetic.

Keywords: Squaring arithmetic, information security, multiplication, 1's complement recoding, algorithm design.

1. Introduction

Squaring is an easier operation than multiplication since half of the single-precision multiplications can be

skipped. This is due to the fact that $t_{ij}=a_i*a_j=t_{ji}$, as shown in Figure 1. Thus, we can modify the standard multiplication procedure to take advantage of this property of squaring operations. So, the efficient squaring is an important role in large arithmetic.

In this paper, a new method is proposed for speeding up the performance of squaring. The proposed method combines the standard squaring method and the 1's complement recoding method. The computational complexity of the proposed

algorithm is $\frac{k^2}{8}$, where k is the bit-length of the exponent.

The rest of this paper is organized as follows. In Section 2, we review the standard squaring method and the complement recoding method. The proposed method is described in Section 3. Finally, we put our conclusions for the proposed method and state some future works.

2. Mathematical Preliminaries

2.1 Squaring Method

Squaring is used to substitute for multiplication because squaring is much more efficient than multiplication [13]. This is due to the fact that $s_{ij}=x_i*x_j=s_{ji}$. The standard squaring procedures are described in Table 1 and in Algorithm 1 [14]. Thus, we can modify the standard multiplication

procedures to take advantage of this property of the squaring operation.

Table 1 The standard squaring procedures.

				x_4	x_3	x_2	x_1
*				x_4	x_3	x_2	x_1
				S_{14}	S_{13}	S_{12}	S_{11}
				S_{24}	S_{23}	S_{22}	S_{21}
				S_{34}	S_{33}	S_{32}	S_{31}
+	S_{44}	S_{43}	S_{42}	S_{41}			
				$2S_{14}$	$2S_{13}$	$2S_{12}$	S_{11}
				2	2	S_{22}	
				S_{24}	S_{23}		
+	S_{44}	$2S_{34}$	S_{33}				
S_8	S_7	S_6	S_5	S_4	S_3	S_2	S_1

Algorithm 1 (The standard squaring algorithm)

Input: Integer $X=(x_n x_{n-1} \dots x_2 x_1)_b$

Output: Integer $S=X^2=(s_{2n} s_{2n-1} \dots s_2 s_1)_b$

begin

$s_i=0$ for $i = 1$ to $2n$

for $i = 1$ to n

$(u, v)_b = s_{i+i-1} + x_i * x_i$

$s_{i+i-1} = v$

for $j = i+1$ to n

$(u, v)_b = s_{i+i-1} + 2 * x_j * x_i + u$

$s_{i+i-1} = v$

endfor

$s_{i+n} = u$

endfor

Return $S=(s_{2n} s_{2n-1} \dots s_2 s_1)_b$

end.

Example 1:

Let $X = 432$, i.e., $n = 3$ and $b = 10$. The squaring procedures of X using Algorithm 1 are shown in Table 1.

2.2 Complement Recoding Method

In 2003, Chang, Kuo and Lin proposed an efficient method to compute the general multiplication operation and for the common-multiplicand multiplication by performing complements. Assume the binary representation of multiplier B is $b_{k-1} b_{k-2} \dots b_1 b_0$. Performing complements is advantageous in the speed up of multiplication computations. The equation is shown as follows.

$$B = \sum_{i=0}^{k-1} b_i \times 2^i, \\ = (10 \dots 0)_{(k+1)\text{bits}} - \bar{B} - 1, \quad (1)$$

where $\bar{B} = \overline{b_{k-1}} \overline{b_{k-2}} \dots \overline{b_0}$,

$$\begin{cases} \bar{b}_i = 0 & \text{if } b_i = 1, \\ \bar{b}_i = 1 & \text{if } b_i = 0, \end{cases} \text{ for } i = 0, 1, \dots, k-1.$$

For example,

$$B = (119)_{10} = (01110111)_2 = (10 \dots 0)_{(8+1)\text{bits}} - \bar{B} - 1 \\ = (10 \dots 0)_{(8+1)\text{bits}} - (10001000)_2 - 1 \\ = 256 - 136 - 1.$$

Clearly, the Hamming weight of B can be reduced. Therefore, the multiplication computations will be speeded up.

Table 1. The procedures of 432^2 using by Algorithm 1.

i	j	$s_{i+i-1} + x_i * x_i$	$s_{i+j-1} + 2 * x_j * x_i + u$	u	v	S_1	s_2	s_3	s_4	s_5	s_6
1	-	$0+2*2$	-	0	4	4	0	0	0	0	0
	2	-	$0+2*3*2+0$	1	2		2				
	3	-	$0+2*4*2+1$	1	7			7			
									1		
2	-	$7+3*3$	-	1	6			6			
	3	-	$1+2*4*3+1$	2	6				6		
										2	
3	-	$2+4*4$	-	1	8				6	8	
		-	-								1
				1	8	4	2	6	6	8	1

3. The Proposed Method

In Section 2, we described a squaring method and a complement recoding method respectively. Now the proposed method is described as follows. We assume there are k bits in X . So X^2 should be calculated by combining a squaring method and a complement recoding method. Therefore, two cases should be discussed as Case 1:

$\text{Ham}(X) > \frac{k}{2}$ and Case 2: $\text{Ham}(X) \leq \frac{k}{2}$. Then the overall computational complexity of X^2 should be represented

$$\frac{1}{2} * \text{the computational complexity of Case 1} \quad (2)$$

or

$$\frac{1}{2} * \text{the computational complexity of Case 2.} \quad (3)$$

The first and the second items " $\frac{1}{2}$ " in Equation (2) and Equation (3) mean the probabilities of $\text{Ham}(X) > \frac{k}{2}$ and $\text{Ham}(X) \leq \frac{k}{2}$. Now, we introduce the computational complexities of Case 1 and Case 2 respectively. At first, we define $\bar{X}$ a 1's complement representation for X .

$$\text{Case 1: } \text{Ham}(X) > \frac{k}{2} :$$

When $\text{Ham}(X) > \frac{k}{2}$ in Case 1, we take 1's complement for X . In other words, it can be represented by $\bar{X}$. It means that $\text{Ham}(\bar{X}) \leq \frac{k}{2}$. We can replace the pattern for X^2 with $[(10\dots0)_{(k+1)\text{bits}} - \bar{X} - 1]^2$, where k is the bit-length of X . For computational complexity, we will discuss it in detail in Section 4.

$$\text{Case 2: } \text{Ham}(X) \leq \frac{k}{2} :$$

When $\text{Ham}(X) \leq \frac{k}{2}$ in Case 2, we square X directly. For computational complexity, we will discuss it in detail in Section 4.

4. Computational Complexity

In Section 3, we discuss the 1's complement applications for X between Case 1 and Case 2. The computational complexities for Case 1 and Case 2 are discussed respectively as follows.

$$\text{Case 1: } \text{Ham}(X) > \frac{k}{2} :$$

The pattern for X^2 can be changed into $[(10\dots0)_{(k+1)\text{bits}} - \bar{X} - 1]^2$. It can be expanded as follows.

$$(10\dots0)_{(2k+1)\text{bits}} - 2 * \bar{X} * (10\dots0)_{(k+1)\text{bits}} - 2 * (10\dots0)_{(k+1)\text{bits}} - (\bar{X})^2 + 2(\bar{X}) + 1 \quad (4)$$

There are six items in Equations (4). We only consider $(\bar{X})^2$ item and other items only considered shift position problem [10, 14]. That is, other five items are not considered for their computational complexity. Then the number of the multiplications for X^2 is $\frac{k^2}{4}$, where k is the bit-length of X .

$$\text{Case 2: } \text{Ham}(X) \leq \frac{k}{2} :$$

Since the Hamming weight of X is smaller than or equal to $\frac{k}{2}$, the number of the multiplications for X^2 is also $\frac{k^2}{4}$, where k is the bit-length of X .

From the above Case 1 and Case 2 depicted in the proposed method, we can therefore get the computational complexity of X^2 as following by using Equation (2) or Equation (3) to be $\frac{1}{2} * \frac{k^2}{4} = \frac{k^2}{8}$. Case 1 or Case 2 can not exist at the same time.

5. Conclusions and Future Works

In this paper, we propose a new method to fast evaluate squaring, which uses a complement recoding method. The computational complexity of the proposed method is $\frac{k^2}{8}$ multiplications, which is

faster than that of the original method $\frac{k^2}{4}$ multiplications, where k is the bit-length of X . We can therefore efficiently speed up the overall performance of squaring.

In the future works, we will do our best to well investigate and properly adopt elliptic curve encrypt methods [3, 7], fast improved Montgomery's method [5, 8], efficient cryptographic hardware design [4, 6] and unified-multipliers method [9, 11]. By exploring these methods and improving some techniques, we will design fast arithmetic algorithms to further decrease the overall computational complexity for information security usages (including key exchange schemes and smart card cryptographic identification applications).

References

- [1] Alkar, A. Z., Sonmez, R., "A Hardware Version of the RSA Using the Montgomery's Algorithm with Systolic Arrays," *VLSI Journal Integration*, vol. 38, no. 2, pp. 299-307, Dec. 2004.
- [2] Chang, C.-C., Kuo, Y.-T., Lin, C.-H., "Fast Algorithms for Common Multiplicand Multiplication and Exponentiation by Performing Complements," *Proceeding of 17th International Conference on Advanced Information Networking and Applications*, pp. 807-811, March 2003.
- [3] Chen, T.-S., "A Threshold Signature Scheme Based on the Elliptic Curve Cryptosystem," *Applied Mathematics and Computation*, vol. 162, no. 3, pp. 1119-1134, March 25, 2005.
- [4] Eslami, Y., Sheikholeslami, A., Gulak, P.G., Masui, S., Mukaida, K., "An Area-Efficient Universal Cryptography Processor for Smart Cards," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 14, no. 1, pp. 43-56, Jan. 2006.
- [5] Katja, S.-S., Semay, O., Takagi, T., "Analysis of Fractional Window Recoding Methods and Their Application to Elliptic Curve Cryptosystems," *IEEE Transactions on Computers*, vol. 55, no. 1, pp. 48-57, Jan. 2006.
- [6] Koc, C. K., "High-Speed RSA Implementation," *Technical Report TR 201*, RSA Laboratories, 1994.
- [7] Nadia, N., Luiza, M. M., "A Review of Modular Multiplication Methods and Respective Hardware Implementations," *INFORMATICA*, vol. 30, no. 1, pp.111-129, April 2006.
- [8] Nedjah, N., Mourelle, L. M., "Four Hardware Implementations for the M-ary Modular Exponentiation," *International Conference on Information Technology: New Generations*, pp. 210-215, April 2006.
- [9] Nedjah, N., Mourelle, L. M., "Three Hardware Architectures for the Binary Modular Exponentiation: Sequential, Parallel, and Systolic Circuits," *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, vol. 53, no. 3, pp. 627-633, March 2006.
- [10] Nibouche, O., Nibouche, M., Ahmed, B., "New Iterative Algorithms for Modular Multiplication," *Signal Processing*, vol. 84, no. 10, Oct. 2004, pp. 1919-1930.
- [11] Sun, D.-Z., Huai, J.-P., Sun, J.-Z., Zhang, J.-W., "Computational Efficiency Analysis of Wu *et al.*'s Fast Modular Multi-Exponentiation Algorithm," *Applied Mathematics and Computation*, In Press, Accepted Manuscript, Available online 27 Feb. 2007.
- [12] Tenca, A. F., Koc, C. K., "A Scalable Architecture for Modular Multiplication Based on Montgomery's

- Algorithm,” *IEEE Transactions on Computers*, vol. 52, no. 9, Sep. 2003, pp. 1215-1221.
- [13] Wu, C.-L., Lou, D.-C., Chang, T.-J., “An Efficient Modular Exponentiation Binary Algorithm Using Shortest Addition Chain,” *Mathematical Meeting and Annual Meeting of the Mathematical Society*, page 41, Dec. 2006.
- [14] Ziya, A. A., Remziye, S., “A Hardware Version of the RSA Using the Montgomery's Algorithm with Systolic Arrays”, *Integration, the VLSI Journal*, vol. 38, no. 2, Dec. 2004, pp. 299-307.